

Série TP 3 : Les bases de PL/SQL

Corrigé

Exercice 1 : Requêtes SQL

Dans cet exercice, nous faisons suite à la série TP2 en utilisant le même schéma de base de données contenant les tables 'departement', 'employe', 'projet' et 'travaille'.

Travail à faire :

1. Écrivez un bloc PL/SQL qui affiche pour chaque département :
 - Le nom du département
 - Le nombre d'employés
 - Le salaire moyen

```
SET SERVEROUTPUT ON;

DECLARE
  TYPE dept_table IS TABLE OF departement%ROWTYPE;
  v_departements dept_table ;
  v_nb_employes NUMBER;
  v_sal_moyen NUMBER;
BEGIN
  -- Récupérer tous les départements
  SELECT * Bulk COLLECT INTO v_departements FROM departement;
  -- Parcourir chaque département
  FOR i IN 1..v_departements.COUNT LOOP
    -- Compter les employés et calculer le salaire moyen
    SELECT COUNT(*), AVG(sal)
    INTO v_nb_employes, v_sal_moyen
    FROM employe
    WHERE numdep = v_departements(i).numdep ;
    DBMS_OUTPUT.PUT_LINE(
      'Département : ' || v_departements(i).nomd ||
      ' | Nombre d"employés : ' || v_nb_employes ||
      ' | Salaire moyen : ' || NVL(ROUND(v_sal_moyen, 2), 0)
    );
  END LOOP;
END;
/
```

2. Écrivez un bloc PL/SQL qui :

- Demande un numéro de département à l'utilisateur
- Affiche le nom du département et son lieu
- Affiche la liste des employés de ce département avec leur nom, fonction et salaire
- Si le département n'existe pas, affiche un message d'erreur

Voici un exemple d'exécution pour la question 2 :

Si le numéro 30 : <pre> === DÉPARTEMENT VENTE === Lieu : CHICAGO ----- • ALLEN INGENIEUR 1600 € • BLAKE MANAGER 2850 € • JAMES COMMERCIAL 950 € • MARTIN INGENIEUR 1250 € • TURNER INGENIEUR 1500 € • WARD INGENIEUR 1250 € ----- Total : 6 employé(s) </pre>	Si le numéro 40 : <pre> === DÉPARTEMENT FABRICATION === Lieu : BOSTON ----- Aucun employé dans ce département. </pre>	Si le numéro 100 : Département 100 introuvable
---	---	--

SET SERVEROUTPUT ON;

DECLARE

```

v_numdep departement.numdep%TYPE := &num_departement;
v_nomd departement.nomd%TYPE;
v_lieu departement.lieu%TYPE;
v_dept_existe NUMBER; -- ou type BOOLEAN

```

BEGIN

```
-- Vérifier si le département existe
```

```
SELECT COUNT(*)
```

```
INTO v_dept_existe
```

```
FROM departement
```

```
WHERE numdep = v_numdep;
```

```
IF v_dept_existe = 0 THEN
```

```
    DBMS_OUTPUT.PUT_LINE('Département ' || v_numdep || ' introuvable.');
```

```
ELSE
```

```
-- Récupérer les infos du département
```

```
SELECT nomd, lieu
```

```
INTO v_nomd, v_lieu
```

```
FROM departement
```

```
WHERE numdep = v_numdep;
```

```
DBMS_OUTPUT.PUT_LINE('=== DÉPARTEMENT ' || v_nomd || ' ===');
```

```

DBMS_OUTPUT.PUT_LINE('Lieu : ' || v_lieu);
DBMS_OUTPUT.PUT_LINE('-----');

-- Afficher les employés directement avec une boucle FOR
FOR emp_rec IN (
    SELECT nom, fonction, sal
    FROM employe
    WHERE numdep = v_numdep
    ORDER BY nom
) LOOP
    DBMS_OUTPUT.PUT_LINE(
        '• ' || emp_rec.nom ||
        ' | ' || emp_rec.fonction ||
        ' | ' || emp_rec.sal);
END LOOP;

-- Vérifier s'il n'y a pas d'employés
IF SQL%ROWCOUNT = 0 THEN
    DBMS_OUTPUT.PUT_LINE('Aucun employé dans ce département.');
```

```

ELSE
    DBMS_OUTPUT.PUT_LINE('Total : ' || SQL%ROWCOUNT || ' employé(s)');
END IF;
END IF;
END;
/

```

Exercice 2 : PL/SQL et structures de contrôle

Soit une base de données dont le schéma est le suivant (les clés primaires des relations sont soulignées).

- Employé (NumEmp NUMBER, NomEmp VARCHAR(20), PrenomEmp VARCHAR(20), FonctionEmp VARCHAR(20), SalaireEmp NUMBER, VilleEmp VARCHAR(20))
- GradeSalaire(Grade VARCHAR(20), SalaireMin NUMBER, SalaireMax NUMBER)
- Département (NumDep NUMBER, NomDep VARCHAR(20), BudgetDep NUMBER, VilleDep VARCHAR(20))
- DirigerDepartement (#NumDep NUMBER, #NumEmp NUMBER, DateDebut DATE, DateFin DATE)
- Projet (NumProjet NUMBER, TitreProjet VARCHAR(100), Montant NUMBER, DateDebut DATE, DateFin DATE)

- Departement_Projet (#NumProjet NUMBER, #NumDep NUMBER, MontantParticipation NUMBER)

Travail à faire :

1. Ecrire le bloc PL/SQL permettant de vérifier que le plus petit salaire minimum et le plus grand salaire maximum de la table **GradeSalaire** sont respectivement inférieur et supérieur au plus petit et au plus grand salaire de la table **Employe**. Afficher le résultat à l'aide du paquetage DBMS_OUTPUT.

```
SET SERVEROUTPUT ON;

DECLARE
    v_min_salaire_grade NUMBER;
    v_max_salaire_grade NUMBER;
    v_min_salaire_employe NUMBER;
    v_max_salaire_employe NUMBER;
BEGIN
    -- Récupérer le plus petit salaire minimum et le plus grand salaire maximum des grades
    SELECT MIN(salairemin), MAX(salairemax)
    INTO v_min_salaire_grade, v_max_salaire_grade
    FROM gradesalaire;

    -- Récupérer le plus petit et le plus grand salaire des employés
    SELECT MIN(salaireemp), MAX(salaireemp)
    INTO v_min_salaire_employe, v_max_salaire_employe
    FROM employe;

    -- Afficher les résultats
    DBMS_OUTPUT.PUT_LINE('=== VÉRIFICATION DES SALAIRES ===');
    DBMS_OUTPUT.PUT_LINE('Salaire minimum des grades : ' || v_min_salaire_grade);
    DBMS_OUTPUT.PUT_LINE('Salaire maximum des grades : ' || v_max_salaire_grade);
    DBMS_OUTPUT.PUT_LINE('Salaire minimum des employés : ' || v_min_salaire_employe);
    DBMS_OUTPUT.PUT_LINE('Salaire maximum des employés : ' || v_max_salaire_employe);
    DBMS_OUTPUT.PUT_LINE('-----');

    -- Vérification des conditions
    IF v_min_salaire_grade < v_min_salaire_employe THEN
        DBMS_OUTPUT.PUT_LINE('Le salaire minimum des grades est inférieur au salaire minimum des employés');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Le salaire minimum des grades n'est pas inférieur au salaire minimum des employés');
```

```

END IF;

IF v_max_salaire_grade > v_max_salaire_employe THEN
    DBMS_OUTPUT.PUT_LINE('✓ OK : Le salaire maximum des grades est supérieur au salaire
maximum des employés');
ELSE
    DBMS_OUTPUT.PUT_LINE('X ERREUR : Le salaire maximum des grades n'est pas
supérieur au salaire maximum des employés');
END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Erreur : Aucune donnée trouvée dans les tables');
END;
/

```

2. Ecrire le bloc PL/SQL qui affiche, à l'aide du paquetage DBMS_OUTPUT, la situation budgétaire des départements sous la forme suivante :

```

Nom du département : NomDep
Budget de département : BudgetDep
Nombre de projets réalisés : Nbr_Projets_Realises
Total des montants de participation aux projets : Total_Montant_Participation

```

```

SET SERVEROUTPUT ON;

DECLARE
    v_nom_dep departement.nomdep%TYPE;
    v_budget_dep departement.budgetdep%TYPE;
    v_nbr_projets NUMBER;
    v_total_participation NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('=SITUATION BUDGÉTAIRE DES DÉPARTEMENTS =');

    -- Parcourir tous les départements
    FOR dept_rec IN (
        SELECT numdep, nomdep, budgetdep
        FROM departement
        ORDER BY nomdep
    ) LOOP
        -- Récupérer le nombre de projets et le total des participations pour ce département
        SELECT COUNT(dp.numprojet), SUM(dp.montantparticipation)

```

```
INTO v_nbr_projets, v_total_participation
FROM departement_projet dp
WHERE dp.numdep = dept_rec.numdep;

-- Afficher les informations du département
DBMS_OUTPUT.PUT_LINE('Nom du département : ' || dept_rec.nomdep);
DBMS_OUTPUT.PUT_LINE('Budget du département : ' || dept_rec.budgetdep);
DBMS_OUTPUT.PUT_LINE('Nombre de projets réalisés : ' || v_nbr_projets);
DBMS_OUTPUT.PUT_LINE('Total des montants de participation aux projets : ' ||
v_total_participation);

-- Calculer et afficher le solde budgétaire
DBMS_OUTPUT.PUT_LINE('Solde budgétaire : ' || (dept_rec.budgetdep -
v_total_participation));
END LOOP;
END;
/
```